⚙️ Rule Engine — Architecture Guide

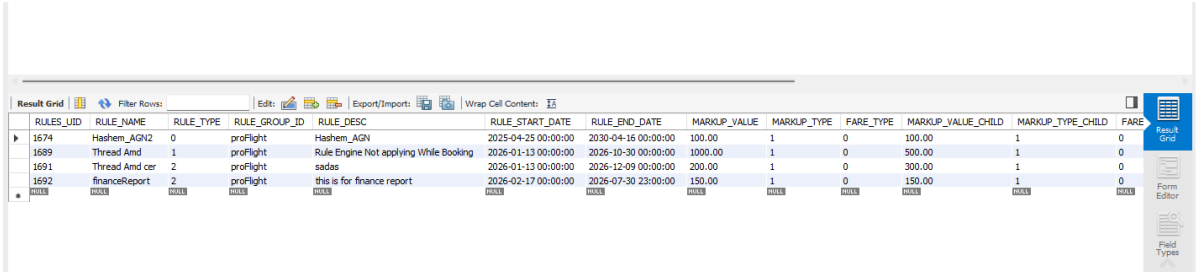
🧩 1. Creating Drools File Generation Flow from Search Booking

🚀 Call: `writeFlightDrlFileOnserverUp`

- This function is responsible for creating Drool file and add it in Redis
- How it works?
  - Check if Drool file exists in Redis using this “flight_rule_text” as a key
  - It firstly checks for this key “flight_rule_text” if exists in Redis
    - ✅ **If exists**
      - Continue to get the search result and after getting the result it calls `applyRuleOnOnwardFlight` to apply the Rule Engine
    - ❌ **If NOT exists**
      - It starts building the Drool file

How to build Drool file?

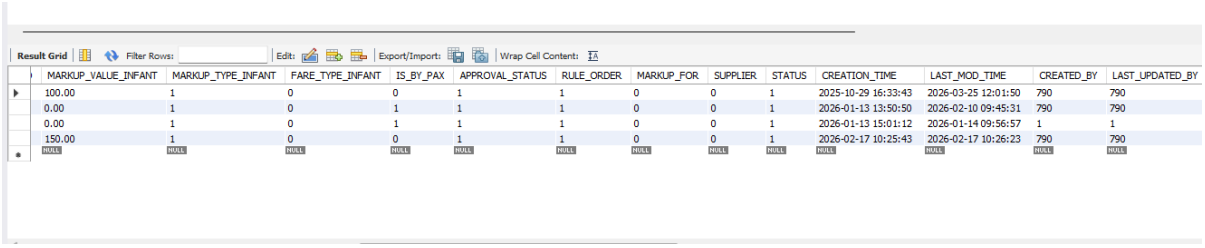
- Call getRulesByRuleGroup → to fetch all active rules from TT_TS_RULE.



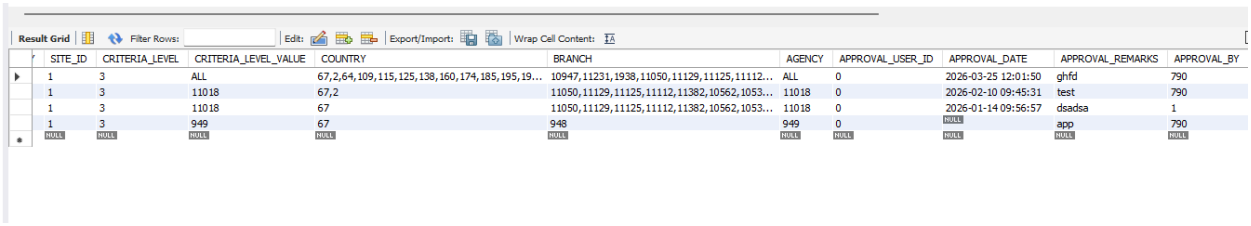
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

RULES_UID	RULE_NAME	RULE_TYPE	RULE_GROUP_ID	RULE_DESC	RULE_START_DATE	RULE_END_DATE	MARKUP_VALUE	MARKUP_TYPE	FARE_TYPE	MARKUP_VALUE_CHILD	MARKUP_TYPE_CHILD	FARE
1674	Hashem_AGN2	0	proFlight	Hashem_AGN	2025-04-25 00:00:00	2030-04-16 00:00:00	100.00	1	0	100.00	1	0
1689	Thread Amd	1	proFlight	Rule Engine Not applying While Booking	2026-01-13 00:00:00	2026-10-30 00:00:00	1000.00	1	0	500.00	1	0
1691	Thread Amd cer	2	proFlight	sadas	2026-01-13 00:00:00	2026-12-09 00:00:00	200.00	1	0	300.00	1	0
1692	financeReport	2	proFlight	this is for finance report	2026-02-17 00:00:00	2026-07-30 23:00:00	150.00	1	0	150.00	1	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

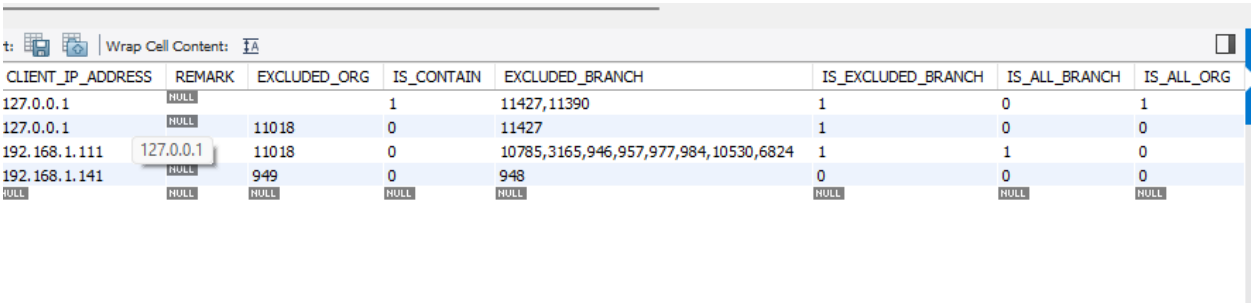
Result Grid | Form Editor | Field Types



MARKUP_VALUE_INFANT	MARKUP_TYPE_INFANT	FARE_TYPE_INFANT	IS_BY_PAX	APPROVAL_STATUS	RULE_ORDER	MARKUP_FOR	SUPPLIER	STATUS	CREATION_TIME	LAST_MOD_TIME	CREATED_BY	LAST_UPDATED_BY
100.00	1	0	0	1	1	0	0	1	2025-10-29 16:33:43	2026-03-25 12:01:50	790	790
0.00	1	0	1	1	1	0	0	1	2026-01-13 13:50:50	2026-02-10 09:45:31	790	790
0.00	1	0	1	1	1	0	0	1	2026-01-13 15:01:12	2026-01-14 09:56:57	1	1
150.00	1	0	0	1	1	0	0	1	2026-02-17 10:25:43	2026-02-17 10:26:23	790	790



SITE_ID	CRITERIA_LEVEL	CRITERIA_LEVEL_VALUE	COUNTRY	BRANCH	AGENCY	APPROVAL_USER_ID	APPROVAL_DATE	APPROVAL_REMARKS	APPROVAL_BY
1	3	ALL	67,2,64,109,115,125,138,160,174,185,195,19...	10947,11231,1938,11050,11129,11125,11112...	ALL	0	2026-03-25 12:01:50	ghfd	790
1	3	11018	67,2	11050,11129,11125,11112,11382,10562,1053...	11018	0	2026-02-10 09:45:31	test	790
1	3	11018	67	11050,11129,11125,11112,11382,10562,1053...	11018	0	2026-01-14 09:56:57	dsadsa	1
1	3	949	67	948	949	0		app	790



CLIENT_IP_ADDRESS	REMARK	EXCLUDED_ORG	IS_CONTAIN	EXCLUDED_BRANCH	IS_EXCLUDED_BRANCH	IS_ALL_BRANCH	IS_ALL_ORG
127.0.0.1			1	11427,11390	1	0	1
127.0.0.1		11018	0	11427	1	0	0
192.168.1.111		11018	0	10785,3165,946,957,977,984,10530,6824	1	1	0
192.168.1.141		949	0	948	0	0	0

📊 Columns Explanation

- **RULE_TYPE**
  - Specify markup - discount - service charge

Rule Type	Value
MARKUP	0
DISCOUNT	1
SERVICE_CHARGE	2

- FARE_TYPE

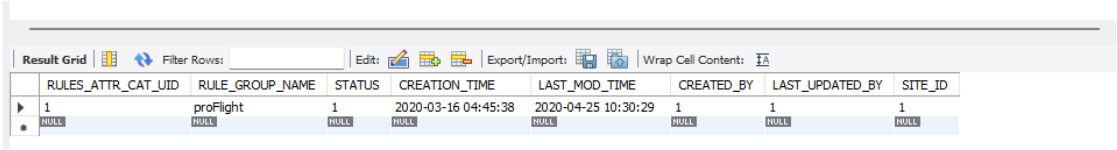
Fare Type	Value
Total Fare	0
Base Fare	1

⚙️ After fetching all active rules , it calls `createDroolRules(rules, ruleGroupName)`

- This function is responsible for creating the Drool file
- To build Drool file, we MUST have all attributes under a given product (proFlight)
- By using this query → SELECT * FROM wonderdb.TT_RULES_ATTR_MAP WHERE RULES_ATTR_CAT_UID = 1; (Fetch all attributes under a specific product)
  - Why RULES_ATTR_CAT_UID = 1 always?
    - As we ONLY have one product in NDC (proFlight)
    - If there are more than one product which can apply Rule Engine on it , so this query will be dynamically

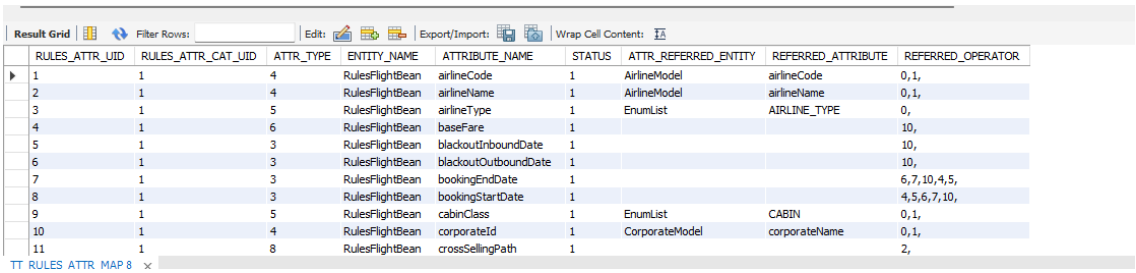
Explaining these attributes, and what tables we use in DB to store them?

- `TT_RULES_ATTR_CATEGORY` → Holds the product name , this table will be useful if we have more than one product in NDC (Hotel - Car - Insurance) as now it holds only the proFlight product ONLY



RULES_ATTR_CAT_UID	RULE_GROUP_NAME	STATUS	CREATION_TIME	LAST_MOD_TIME	CREATED_BY	LAST_UPDATED_BY	SITE_ID
1	proFlight	1	2020-03-16 04:45:38	2020-04-25 10:30:29	1	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

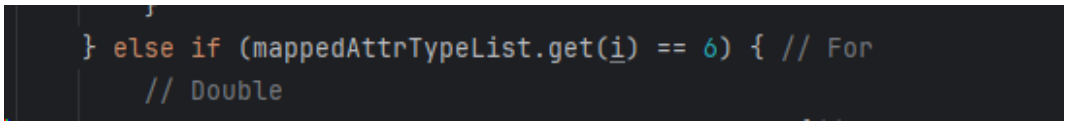
- `TT_RULES_ATTR_MAP` → contains all attributes under a specific product which is proFlight



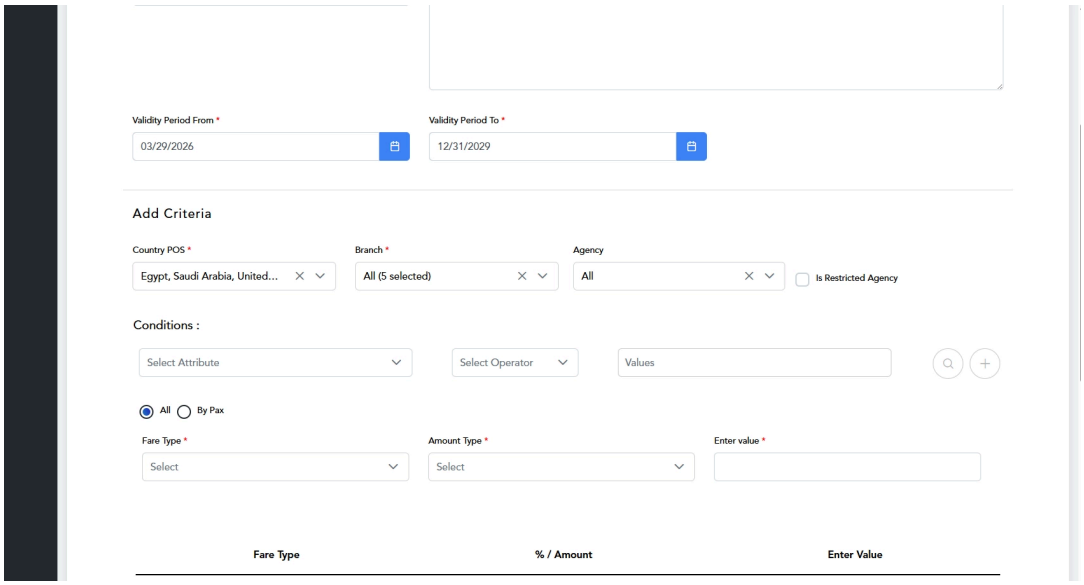
RULES_ATTR_UID	RULES_ATTR_CAT_UID	ATTR_TYPE	ENTITY_NAME	ATTRIBUTE_NAME	STATUS	ATTR_REFERRED_ENTITY	REFERRED_ATTRIBUTE	REFERRED_OPERATOR
1	1	4	RulesFlightBean	airlineCode	1	AirlineModel	airlineCode	0,1,
2	1	4	RulesFlightBean	airlineName	1	AirlineModel	airlineName	0,1,
3	1	5	RulesFlightBean	airlineType	1	EnumList	AIRLINE_TYPE	0,
4	1	6	RulesFlightBean	baseFare	1			10,
5	1	3	RulesFlightBean	blackoutInboundDate	1			10,
6	1	3	RulesFlightBean	blackoutOutboundDate	1			10,
7	1	3	RulesFlightBean	bookingEndDate	1			6,7,10,4,5,
8	1	3	RulesFlightBean	bookingStartDate	1			4,5,6,7,10,
9	1	5	RulesFlightBean	cabinClass	1	EnumList	CABIN	0,1,
10	1	4	RulesFlightBean	corporateId	1	CorporateModel	corporateName	0,1,
11	1	8	RulesFlightBean	crossSellingPath	1			2,

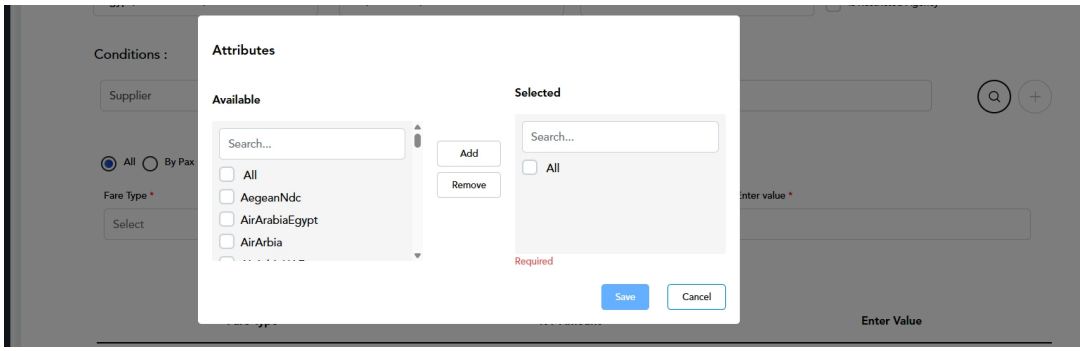
📊 What Columns inside TT_RULES_ATTR_MAP

- **RULE_GROUP_NAME**
  - `proFlight`
  - Useful for multi-products (Hotel / Car / etc)
- **ATTR_TYPE**
  - Data Type
  - You can see that Data Type here is about numerical values (4 - 3 - 5 - etc)
  - These numerical values are NOT mapped using ENUM in the code, BUT it is commented in the code. Like this



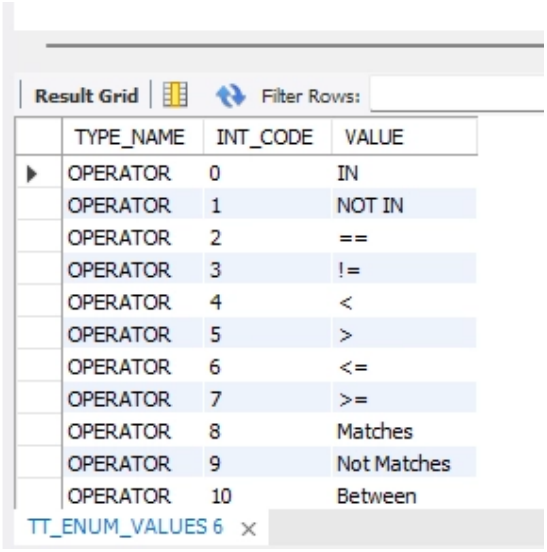
    - Here mappedAttrTypeList is a List<Integer> in the code , and every index in this list contains a specific numeric value which represents the ATTR_TYPE column.
    - Suggestion → to convert this using ENUM, whether in the code or in DB.
- **ATTR_REFERRED_ENTITY**
  - Some attributes have a table in DB , Like supplier attribute , it has TT_TS_SUPPLIER in DB
  - Why this column useful?
    - When adding a new rule , and adding a new condition to this rule. You need to select an attribute and select values under this attribute.
    - So ENTITY_NAME column helps in fetching all values under the selected attributerom UI





- **REFERRED_OPERATOR**
  - Each attribute MUST have a value that defines the operator should be applied (IN - NOT IN - EQUAL - etc)
  - Why these operators (IN - NOT IN - etc)?
    - These operators define the way that this attribute will be represented in DRool file
    - As IN operator will be represented as a list
  - The problem in this column is that these operators are not stored like (IN - NOT IN - etc), but they are stored as a numerical values from 0 to 10
  - BUT this problem is solved in the code by mapping these numerical values to their text value (example → 0 is mapped to IN). This mapping

is done using TT_ENUM_VALUES table as this table stores all the operators and their mapping values



Result Grid | Filter Rows:

TYPE_NAME	INT_CODE	VALUE
OPERATOR	0	IN
OPERATOR	1	NOT IN
OPERATOR	2	==
OPERATOR	3	!=
OPERATOR	4	<
OPERATOR	5	>
OPERATOR	6	<=
OPERATOR	7	>=
OPERATOR	8	Matches
OPERATOR	9	Not Matches
OPERATOR	10	Between

TT_ENUM_VALUES 6

📦 Inside createDroolRules function

- First add these packages and libraries in DRool file
  - package com.travel.ruleengine.model.airline

    import com.tt.ts.ruleengine.model.airline.RulesCommonBean
    import com.tt.ts.ruleengine.model.airline.RulesFlightBean
    import java.util.Arrays
    import java.util.List
    import java.util.ArrayList
    import java.util.HashSet
  - Then it calls `createRuleCondition` → This function is responsible for creating the conditions in Drool file based on all active rules from TT_TS_RULES table

🔁 How `createRuleCondition` works?

- Append Common Data first
  - ruleType

    - markup / discount / service charge
  - ticketingPeriod
  - Country
  - branches
  - agencies

```
rule "Thread Amd cer_1691"
when
      $RulesCommonBean:RulesCommonBean( ruleType == "discOrSc" &&  ticketingPeriod >= "13-Jan-2026 00:00" && ticketingPeriod < "10-Dec-2026 00:00"  &&
countryId ==  "67" && new HashSet(Arrays.asList( "10785", "3165", "946", "957", "977", "984", "10530", "6824")) not contains (branchId) &&  null !=
(agencyId) && new HashSet(Arrays.asList( "11018")).contains(agencyId))
```

- Append Conditions
  - Based on attributes on the given rule
  - These conditions are written in **static way in code**
  - The code checks for every attribute and write the condition on Drool file in a specific way (static in the code)
  - Like this



```
String[] criteriaArray = ruleCriteria.getCriteriaValue().split( regex: "|");

if ("rbd".equalsIgnoreCase(attributeName) || "airlineType".equalsIgnoreCase(attributeName) || "journeyType".equalsIgnoreCase(attributeName)) {
    rulesFlight.append(attributeName + "." + Operator.CONTAINS.getOperatorString() + "(\"" + criteriaArray[0].trim() + "\")");
    if (criteriaArray.length > 1) {
        for (int index = 1; index < criteriaArray.length; index++) {
            rulesFlight.append(" || ");
            rulesFlight.append(attributeName + "." + Operator.CONTAINS.getOperatorString() + "(\"" + criteriaArray[index].trim() + "\")");
        }
    }
}
else if ("flightNo".equalsIgnoreCase(attributeName)) {
    rulesFlight.append(attributeName + "." + Operator.CONTAINS.getOperatorString() + "(");
    rulesFlight.append(criteriaArray[0].trim() + ")");
    if (criteriaArray.length > 1) {
        for (int index = 1; index < criteriaArray.length; index++) {
            rulesFlight.append(" || ");
            rulesFlight.append(attributeName + "." + Operator.CONTAINS.getOperatorString() + "(");
            rulesFlight.append(criteriaArray[index].trim() + ")");
        }
    }
}
else {
```

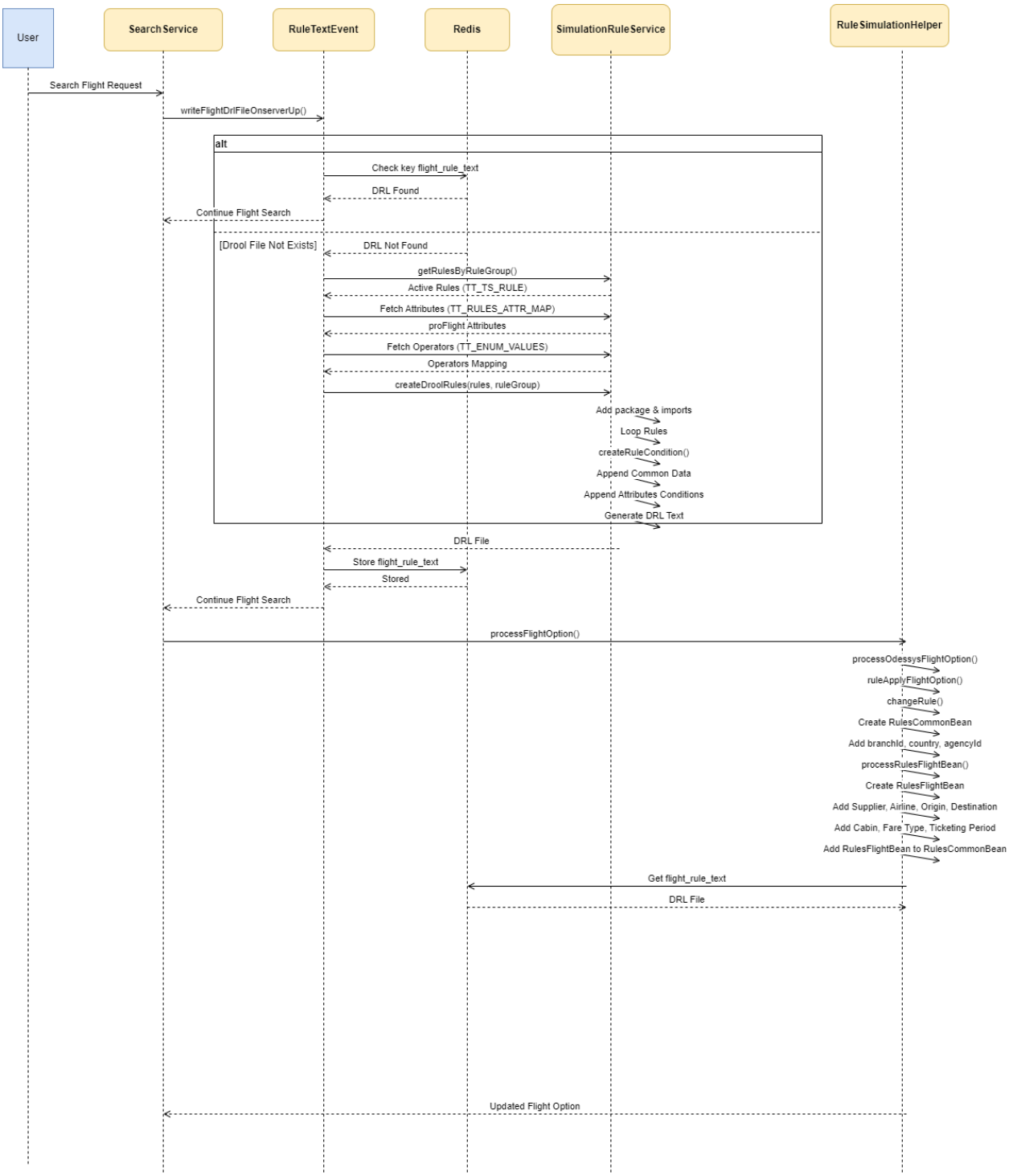
After Processing All Rules

- Store generated Drool file in Redis and flight_rule_text as a key

What happen after creating DRool file from Search Booking?

🔄 Call: `processFlightOption`

- This function is responsible for applying the rule engine on the flights
- Inside this function it calls `processOdeysysFlightOption`
- Then it calls `ruleApplyFlightOption`
  - It calls `changeRule` function
    - This function is responsible for adding (branchId - country - agencyId) and storing them into RulesCommonBean. Why this object specific (RulesCommonBean)? As KieSession interacts with Drool file to apply rules using RulesCommonBean object
  - After adding the prev data, it calls `processRulesFlightBean`
    - This function is responsible for taking specific data from FlightOption (Flight), and put it into RulesFlightBean. Why RulesFlightBean that holds the data from FlightOption? As the conditions' attributes are stored in RulesFlightBean in Drool file
    - Note that after adding the specific data from FlightOption into RulesFlightBean, then we add RulesFlightBean into RulesCommonBean. As KieSession interacts with Drool file by RulesCommonBean
  - After calling `processRulesFlightBean`
    - RulesCommonBean with other data (TicketingPeriod - Fares (Total Fare - Base Fare))
  - After preparing RulesCommonBean with all data , then calling `fireRule` and this function is responsible for applying rule engine (Markup - Discount - Service Charge) on that FlightOption


User
Search Service
RuleTextEvent
Redis
SimulationRuleService
RuleSimulationHelper
Search Flight Request
writeFlightDrlFileOnserverUp()
alt
Check key flight_rule_text
DRL Found
Continue Flight Search
[Drool File Not Exists]
DRL Not Found
getRulesByRuleGroup()
Active Rules (TT_TS_RULE)
Fetch Attributes (TT_RULES_ATTR_MAP)
proFlight Attributes
Fetch Operators (TT_ENUM_VALUES)
Operators Mapping
createDroolRules(rules, ruleGroup)
Add package & imports
Loop Rules
createRuleCondition()
Append Common Data
Append Attributes Conditions
Generate DRL Text
DRL File
Store flight_rule_text
Stored
Continue Flight Search
processFlightOption()
processOdessysFlightOption()
ruleApplyFlightOption()
changeRule()
Create RulesCommonBean
Add branchId, country, agencyId
processRulesFlightBean()
Create RulesFlightBean
Add Supplier, Airline, Origin, Destination
Add Cabin, Fare Type, Ticketing Period
Add RulesFlightBean to RulesCommonBean
Get flight_rule_text
DRL File
Updated Flight Option

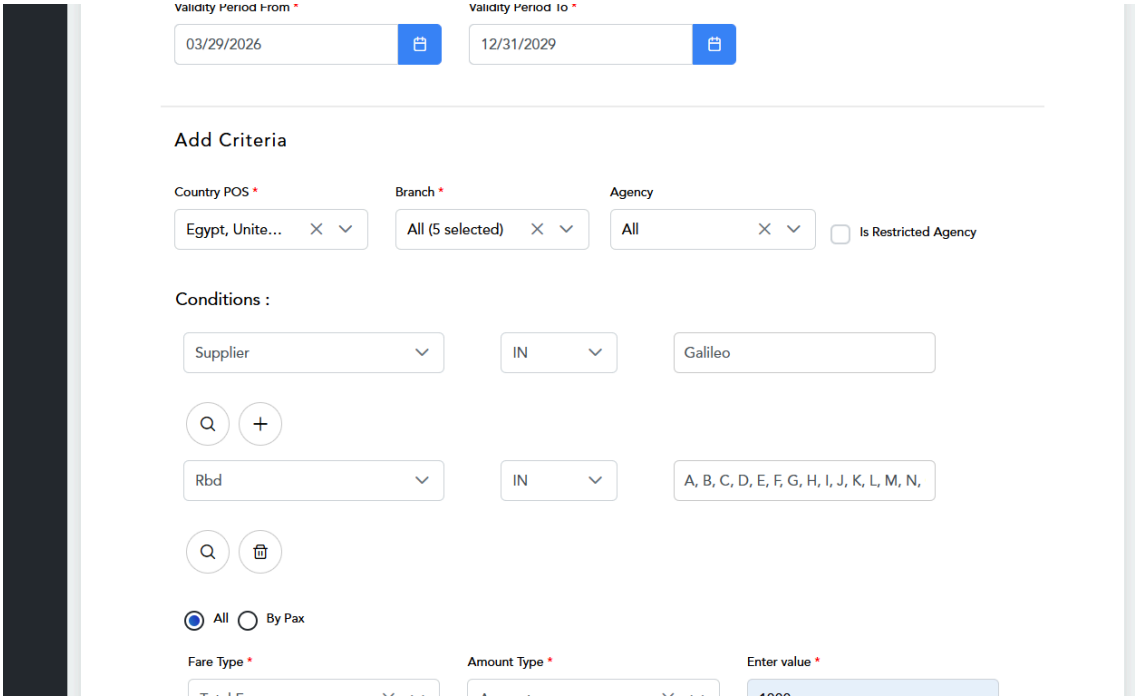
2. Create Drools File Generation Flow from CRUD operations

➕ Add Rule — Full Lifecycle

- **User Selects Criteria**

  In the Rule UI, the user picks:

  - **Attribute** (e.g. Supplier)
  - **Operator** (e.g. IN)
  - **Value** (e.g. "Galileo")



- **Call** `buildBaseRules`
  - This function is responsible to convert the AddingRuleDTO INTO Rule object so it can be added into TT_TS_RULES table
- call `createRule`
  - save it to TT_TS_RULES table
  - Call `makeRuleAsyncText`

- This function is responsible to recreate Drool file after adding a new rule , in addition to updating Drool file in Redis
- Inside this function `createDroolRules` is called
  - Remember that `createDroolRules` is responsible to fetch all active rules from TT_TS_RULES table , and represent these rules in DRool file
  - But the problem is when adding any rules ,the creating rules is inactive by default, and when creating the Drool file it fetches all active rules, so this is a useless operation
- Update Cache
- `processRuleTextToProduct`
  - This function for updating the RULE_LAST_UPDATED (Data Type for this column is Date) column in TT_TS_PRODUCTS
  - Why we need to update RULE_LAST_UPDATED , this helpful when checking for KieSession whether we can use it to apply rule engine OR generate a new KieSession, how this works?
    - By Checking RULE_LAST_UPDATED and the Redis TTL for this key “flight_rule_text”, we can determine if the KieSession is still be available or we need to generate a new one, How?
    - If the RULE_LAST_UPDATED is after KieSession.LastUpdatedTime, so we need to generate a new KieSession from KieBuilder





```
//new Thread(() -> {
    Long k1 = System.currentTimeMillis();
    logHelper.startExternalSystemCallINFO(REDIS,getRuleTextFromCache(),
            LogHelper.LogContext.builder()
                    .add(QUERY_METHOD,"KieSessionService.writeRuleTextFlight")
                    .add(RULE_LAST_UPDATE_IN_DB, ruleLastUpdated !=null ? ruleLastUpdated.toInstant().toString() : null )
                    .add(KIE_SESSION_LAST_UPDATED,kieSessionLastUpdated != null ? kieSessionLastUpdated.toInstant().toString()
    boolean flagTextRule = KieSessionService.writeRuleTextFlight(getRuleTextFromCache());
    //boolean flagTextRule = KieSessionService.writeRuleTextFlight(productModal.getRuleText());
    if (flagTextRule) {
        KieSessionService.kieSessFlight = null;
        KieSessionService.getKieSessionFlight();
        KieSessionService.kieSessionLastUpdatedFlightSearch.put(requestBean.getFlightSearchKey(), new Date());
```

- Call `createNewBatch` → this is responsible to add the new Drool file (compressed) into TT_TS_BATCH_RULE table. Why this phase?
  - Storing the Drool File was used before in reCalculateDiscount in Itinerary page, but this part is no longer available in NDC. So we have a useless table in DB (TT_TS_BATCH_RULE)

Other CRUD Actions (update status - edit rule - delete)

- Update the status Action in TT_TS_RULES table
  - Update TT_TS_PRODUCTS
  - `makeRuleAsyncText` → to update Drool file
    - `createDroolRules` → Regenerate Drool File
    - update Redis
    - `updateBatchRule` → this for update the existing compress Drool file or create other one
    - This function is useful here as when fetching the active rules , there will be a new active rule in DB from the one you activated in UI.
- NOTE that all other actions has the same flow like updating status

---

3. Current Problem

> 🚨 **Root Cause** → The current implementation of the flight rules engine has a fundamental architectural issue: the logic that builds Drool condition expressions is hardcoded in Java, keyed on attribute names. When a developer needs to support a new rule attribute, they must write a new conditional branch in the source code, submit it for review, and trigger a full deployment cycle —

> even though the change is entirely a business configuration concern that carries no new logic.
> This creates an unnecessary and costly dependency between business rule configuration and the software delivery pipeline.

The Hardcoded Pattern

```
if (attributeName.equals("rbd")) {
    condition = "rbd.contains(\"" + value + "\")";
} else if (attributeName.equals("flightNo")) {
    condition = "flightNo.contains(" + value + ")";
}
```

Impact

Problem	Description
⛓️ **Not Scalable**	Every new attribute requires a developer, code review, and a full deployment
🔒 **Tightly Coupled**	Business rule config is buried inside application code, not configuration
Slow to Change	What should be a DB update becomes a Code Change → Test → Deploy → Release cycle

---

4. Proposed Solution

The existing TT_RULES_ATTR_MAP table already contains two columns that, together, fully describe how a Drool condition should be built for any attribute:

- REFERRED_OPERATOR defines the logical operation — for example IN, NOT IN, equals, less than, or greater than.
- ATTR_TYPE defines the data type of the attribute's value — for example String, Number, or Date.

The insight that drives this solution is that the combination of these two existing columns is sufficient to determine the correct Drool expression for any attribute,

without any need for attribute-name-specific logic in Java and without any schema changes. The information required to generate the expression already exists in the database — it simply has not been used that way

⚠️ The Gap in Current Design

🔴 What `REFERRED_OPERATOR` Column Handles

It defines:

```
WHAT operator to use
```

Examples:

- IN
- NOT IN
- =
- List
- <

🔴 What It DOES NOT Handle

It does NOT define:

```
HOW the condition should be built in Drools
```

📌 Example

Case 1: supplier

```
REFERRED_OPERATOR = IN
```

Output:

```
new HashSet(...).contains(supplier)
```

Case 2: flightNo

```
REFERRED_OPERATOR = IN
```

BUT output is:

```
flightNo.contains(872) || flightNo.contains(808)
```

👉 Same operator (`IN`)

👉 Different behavior

How will the implementation be?!

- The solution replaces the attribute-name-based if/else logic with a registry of expression builders, where each builder is responsible for one specific (operator, data type) combination.
- When the system needs to generate a condition for an attribute, it reads the attribute's REFERRED_OPERATOR and ATTR_TYPE from the database, looks up the matching builder in the registry, and delegates the expression construction entirely to that builder.
- Because every attribute already has these two columns populated, the system can immediately determine the correct builder for any attribute — existing or new — without any Java code modification. Adding a new attribute to the rule system becomes a database operation only

How Operators Map to Expression Structures

Each operator has a natural expression structure, and the data type parameterises how values are formatted within that structure. The following table illustrates the mapping across the operators currently in use:

Operator	Data Type	Expression Structure	Behaviour
IN	String	Set containment check	Checks whether the field value is present in a set of string literals. The set is built from the comma-separated values stored in the rule configuration.
IN	Number	Set containment check	Same structural check as IN/String, but values are numeric literals. No quoting is applied.
NOT IN	String	Negated set containment	Logical inverse of IN/String. The condition is true when the field value is absent from the set.
NOT IN	Number	Negated set containment	Logical inverse of IN/Number.
EQUALS	String	Direct equality check	Checks whether the field value is exactly equal to a single string literal.
EQUALS	Number	Direct equality check	Checks whether the field value is numerically equal to a single value.
LESS THAN	Number	Numeric comparison	Checks whether the field value is strictly less than the threshold value.
GREATER THAN	Number	Numeric comparison	Checks whether the field value is strictly greater than the threshold value.
BETWEEN	Number	Range check	Checks whether the field value falls within an inclusive lower and upper bound, both stored in the rule configuration.
BETWEEN	Date	Date range check	Same range structure as BETWEEN/Number, with values parsed as dates.

This table is not a fixed list — it is a description of the registry's current contents. New (operator, data type) combinations can be added to the registry as new

requirements emerge, without touching any existing builder or any other part of the system.